UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CIENCIAS

Práctica 7
Análisis Semántico y Verificación de Tipos.

ALUMNO

Gallegos Cortes José Antonio - 320316566
Angel Maldonado Gerardo de Jesus - 320188268

PROFESOR

Victor Germán Mijangos.

AYUDANTES

Adrián Martínez Manzo
Teresa Becerril Torres

ASIGNATURA

Compiladores.

Descripción breve del programa

Nuestro programa para esta práctica realiza los siguiente:

1. Recibe como entrada un código escrito en nuestro lenguaje basado en Morse.
2. El lexer y parser (que adaptamos de la práctica anterior) generan un Árbol de Sintaxis Abstracta puro, no realizan validaciones sobre la marcha.
3. Le pasamos ese árbol a un nuevo módulo independiente llamado `SemanticAnalyzer` que lo recorre de manera recursiva nodo por nodo.
4. Durante el recorrido, vamos guardando las variables en una Tabla de Símbolos, verificando si ya existen en el ámbito (scope) actual o si no han sido declaradas.
5. Verificamos los tipos de dato en asignaciones y operaciones matemáticas. Si sumamos un entero con un flotante, inyectamos un nodo `Cast_IntToFloat` en el árbol de manera automática para no perder la semántica.
6. Al final, si hay errores semánticos, los imprimimos todos de un jalón indicando línea y columna. Si todo está correcto, imprimimos el AST completo y decorado con sus tipos finales.

La idea fue mantener el parser lo más limpio posible y delegar toda la chamba de validación a este analizador semántico.

Mini Tests

Preguntas de Diseño

1. Gestión de Símbolos en C/C++

Interfaz de Funciones: ¿Cuál será la diferencia en la firma (argumentos y valor de retorno) entre tu función para definir una variable y tu función para buscar/modificar una existente?
Para definirla debe de recibir al menos un nombre, tipo y el ambito actual, tipo si s un bool o si necesita un puntero. La función debe recibir el nombre y el ambito más interno al más externo, este regresa un puntero al simbolo.

Manejo de Memoria: Si usas structs o classes para los símbolos, ¿en qué momento exacto liberarás la memoria dinámica (ej. llamadas a free o delete) de una tabla de símbolos local al salir de un bloque?
Debe liberarse cuando se abandona el bloque al que pertenece ese ambito, es decir al cerrar una llave o al terminar la construccion sintactica.

2. Análisis de Ámbito (Scope)

Lógica de Shadowing: Si el usuario declara `int x` en global y `float x` en local, ¿cómo garantizarás algorítmicamente que al referenciar `x` dentro del bloque de C/C++ se obtenga la versión local sin borrar la global de la tabla?
Manteniendo una tabla por ambito o una pila de tablas de simbolos, al declararse se inserta solamente en la tabla del ambito actual, se busca del ambito más interno hacía afuera, encontrando local primero.

Protección de Redeclaraciones: En el código `{ int a; float a; }`, ¿cómo detectarás que es un error semántico? Describe qué operación de la tabla lanzarías antes de la inserción.
Al insertar el segundo simbolo porque consultaremos la tabla del ambito actual para verificar si ya existe un identificador con el mismo nombre, si es así se lanza un error de redeclaracion.

3. Chequeo de Tipos y Decoración del AST

Estructura del Nodo: Describe qué campos adicionales añadirás a tus structs/classes del AST para almacenar el "tipo de dato evaluadoüna vez procesado el nodo.
Al menos un campo para almacenar el tipo semantico resultante del nodo, como unn INT, FLOAT, ETC. También un campo booleano que valide la semantica o cuando haya un error.

Validación Operativa: Para una operación como `*`, ¿cómo iterarás sobre los hijos del nodo AST en C/C++ para validar que ambos operandos son numéricos antes de asignar el tipo resultante al nodo padre?
Analizndo recursivamentre a los dos hijos y obteniendo el tipo decorado de cada uno, luego verificamos que ambos pertenezcan al conjunto de tipos numericos permitidos.

4. Lógica de Promoción Explícita/Implícita

Nodos de Cast: Cuando ocurra una promoción válida (ej. entero a flotante), ¿inyectarás un nuevo "nodo de conversión"(cast node) transparente en el AST o solo actualizarás el tipo del nodo original? Justifica las ventajas computacionales de tu elección.
Inyectaría un nodo explicito de conversión porque así el arbol represneta mejor la transformación que hubo, en fases posteriores podriamos ver más claramente donde hubo conversión, evitamos ambiguedades y es más fácil de mantener.

Control Activo de Pérdida: ¿Cómo detectarás y restringirás en C/C++ el intento de asignar un flotante a un entero (`int a = 3.14`) si las reglas de tu lenguaje lo prohíben?
Comparando el tipo del lado derecho con el tipo del lado izquierdo, si el tipo del lado derecho es un flotante y el del lado izquierdo es un entero, se lanza un error semántico y no les insertamos ningun cast.

5. Motor de Validación Semántica

Estrategia de Recorrido: ¿Vas a usar recursión estándar anidada en C u optarás por implementar el patrón Visitor con clases en C++? Argumenta en términos de escalabilidad de código.
El patrón visitor es más complicado que la recursión anidada, pero es más esacalable y mantenible si mantenemos muchas operaciones semanticas diferentes sobre los mismos nodos. Para el proyecto quizá la usemos, depende cuan grande lo vayamos a hacer.

Contención de Errores: En lugar de tirar un exit(1) al primer error de compilación, ¿qué estructura utilizarás en C/C++ (ej. una lista global de strings, o un array de logs) para acumular y reportar múltiples fallos al final del procesamiento?
Un array de logs con una entrada del mensaje y tal vez una referencia a la linea del error, seria un arreglo dinamico.

Implementación

Para la implementación de la estructura principal, decidimos usar una estrategia de recursión estándar anidada en lugar de implementar el patrón Visitor completo de golpe. Nos pareció que, por ahora, recorrer el árbol llamando a funciones específicas como `visit_block`, `visit_declaration`, etc., era más fácil de entender y nos permitía avanzar sin perder el control de la arquitectura. Si en el futuro el proyecto crece, será fácil migrarlo a un Visitor.

Para la tabla de símbolos usamos un `std::vector<std::unordered_map<std::string, Symbol>>` que funciona prácticamente como una pila. Cuando entramos a un nodo `Block` (las llaves), hacemos `enter_scope()` agregando un nuevo mapa al final del vector, y cuando salimos hacemos `leave_scope()` para borrarlo. Cuando necesitamos buscar un símbolo, las búsquedas iteran de reversa, buscando del mapa más profundo (el scope local) hacia afuera (hasta el global), garantizando el funcionamiento de reglas como el *shadowing*.

Otra cosa importante que implementamos es la inyección de nodos cast. Cuando evaluamos una operación binaria (por ejemplo, una suma entre un `int` y un `float`), revisamos los tipos de los hijos. Si detectamos que uno es `int` y necesitamos promoverlo para no perder precisión, creamos explícitamente un nuevo nodo `Cast_IntToFloat`. En ese momento sacamos al hijo original de su posición temporalmente usando punteros, lo metemos como hijo dentro del nuevo nodo Cast, y reacomodamos el Cast donde estaba originalmente el hijo. De este modo el AST final refleja explícitamente que hubo una conversión implícita por debajo.

Y para el manejo de los errores, hicimos una lista dinámica de errores (`std::vector<SemanticError>errors_`) que va juntando todos los mensajes, asociándolos a la línea y la columna de código en las que ocurrieron, para luego reportarlos bonito al final.

Pruebas realizadas

Para comprobar que nuestro analizador funcionaba como esperábamos, usamos archivos `.morse` de la carpeta de ejemplos. Algunos están hechos para fallar a propósito y así corroborar que los errores son atrapados.

Por ejemplo, intentamos validar un archivo con una redeclaración en el mismo bloque:

```
./validator examples/redecl.morse
```

El resultado atrapó el error perfectamente, indicando que el símbolo ya existía:

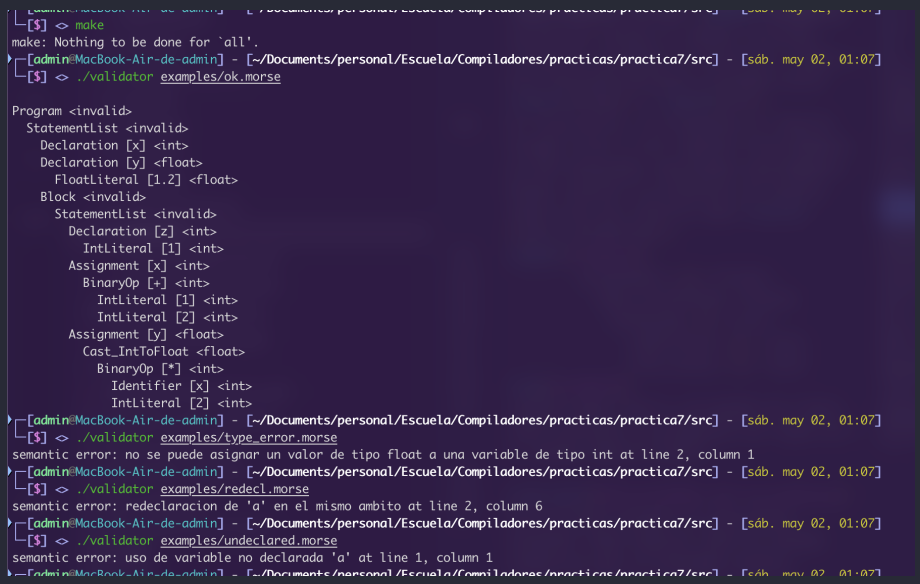
```
semantic error: redeclaracion de 'a' en el mismo ambito at line 2, column 1
```

Y con un error de asignación de tipos, como intentar asignar un flotante a una variable entera (lo cual se prohíbe para no truncar la información):

```
./validator examples/type_error.morse
```

El validador se dio cuenta y lo bloqueó con este mensaje de salida:

```
semantic error: no se puede asignar un valor de tipo float a una variable
de tipo int at line 2, column 1
```



Además, probamos un programa completamente válido (`examples/ok.morse`) donde el código contiene variables anidadas en distintos scopes e hizo multiplicaciones cruzadas entre `float` e `int`. Al ejecutarlo, el programa no reportó ningún error y nos regresó directamente el AST decorado, mostrando dónde inyectó la promoción de manera implícita con nuestro nodo `Cast_IntToFloat`:

Dificultades con la práctica

Fue un poco retador al principio entender cómo separar todo el código semántico que ya habíamos puesto directamente en `parser.y` durante la práctica pasada y moverlo a una clase C++ totalmente independiente. También nos perdimos un buen rato al intentar inyectar los nodos Cast en el AST puro sin romper los punteros ni dejar basura en la memoria, pero al final lo logramos resolviendo el paso de punteros usando funciones que liberaran la propiedad de los nodos temporalmente (`release`) y los reacomodaran con cuidado.